

Intro

As I learned C this spring semester, I found myself becoming more interested in systems programming and the ideas that shape how low level languages work. At the same time, I have spent the last couple of years watching a lot of Linux focused content, and that space has many discussions about the relationship between C and Rust. These conversations made me curious about how a modern systems language approaches memory safety compared to a language like C that gives the developer complete freedom. This project grew out of that curiosity. I wanted to explore the differences between C's manual memory model and Rust's ownership based approach, and I wanted to do so by implementing the same data structure in both languages. I chose a dynamic array because it is simple enough to build from scratch, yet complex enough to reveal how each language handles memory.

Project Goals and Methods

My main goal was to understand how Rust prevents the kinds of memory errors that C allows. I wanted to see how another low level language handled memory safety, and I was especially interested in Rust's idea of allowing unsafe operations only when the developer literally types the word `unsafe` (Rust Team, 2024). I found this approach to be clever because it does not hide the danger. Instead, it forces the developer to acknowledge that they are stepping outside the language's guarantees.

To carry out the project, I used Visual Studio Code as my development environment because it is a reliable tool I have used for years. I used GCC to compile my C code since it came preinstalled on my system. For Rust, I used Cargo to create and manage the project. I wrote an unsafe version of my Rust code that reproduced the C bugs, and then I created a safe version by removing the unsafe keyword. Rust rejected the safe version immediately, which was an important part of the comparison

Implementing the Dynamic Array in C

C does not provide a dynamic array in its standard library, so I had to build one myself (Free Software Foundation, 2024). This required manually allocating memory, tracking the size and capacity, resizing the buffer when needed, and freeing the memory at the end. This process gives the developer a lot of control, but it also leaves room for mistakes. To highlight this, I intentionally introduced three common memory bugs into my demo. These were a use after free, a double free, and a buffer overflow. I chose these bugs because they are simple errors that could easily occur in real programs through ordinary mistakes or miscommunication, and C will compile them without complaint.

When I ran the buggy C code, I observed some interesting behavior. The use after free always caused a segmentation fault, which is what I expected. The double free and buffer overflow behaved differently. Both would trigger case 4 in my menu driven program, which suggests that the memory for that case was located near the corrupted memory region. This was a reminder that undefined behavior does not always crash. It can do anything, including running the wrong part of the program.

```
void pushToDynamicArray(DynamicArray *arrPtr, int value)
{
    if (arrPtr->size >= arrPtr->maxSize)
    {
        int newMaxSize = arrPtr->maxSize * 2;
        int *newData = realloc(arrPtr->data, newMaxSize * sizeof(int));
        if (!newData) {
            printf("realloc failed\n");
            return;
        }
        arrPtr->data = newData;
        arrPtr->maxSize = newMaxSize;
    }
    arrPtr->data[arrPtr->size] = value;
    arrPtr->size++;
}
```

My implementation for “pushToDynamicArray”, followed by the struct in the header file.

```
typedef struct
{
    int *data;
    int size;
    int maxSize;
} DynamicArray;
```

Reproducing the Bugs in Rust

To recreate the same bugs in Rust, I had to break apart the safe abstractions that `Vec<T>` normally provides. Rust’s standard library is designed to prevent misuse, so I had to manually extract raw pointers, reconstruct vectors from raw parts, and dereference pointers directly. All of these actions required the `unsafe` keyword. As I worked through this, I realized that Rust makes it difficult to misuse memory by accident. You can still do dangerous things, but only if you explicitly opt out of safety. This is very different from C, where the dangerous path is the default.

The safe version of my Rust code was simply the unsafe version with the `unsafe` keyword removed. Rust’s compiler rejected it immediately. This happened because Rust’s ownership, borrowing, and lifetime rules prevent the exact operations that caused bugs in C. Rust would not let me dereference a raw pointer or reconstruct a vector from raw parts without `unsafe`. These

operations violate Rust’s safety guarantees, so the compiler stops them before the program ever runs.

```
{
    let mut vector = vec![0, 0];
    let ptr = vector.as_mut_ptr();
    let oldVectorLength = vector.len();
    let oldVectorCapacity = vector.capacity();
    drop(vector);
    let firstVec = unsafe { Vec::from_raw_parts(ptr, oldVectorLength,
oldVectorCapacity) };
    drop(firstVec);
}
```

A demonstration of forcing rust to have a double free error.

```
error[E0133]: call to unsafe function `Vec::<T>::from_raw_parts` is unsafe and requires unsafe block
--> src/bin/safe.rs:34:32
|
|   let firstVec = Vec::from_raw_parts(ptr, oldVectorLength, oldVectorCapacity); // no unsafe == rust refuses to compile
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ call to unsafe function
|
= note: consult the function's documentation for information on how to avoid undefined behavior
```

A demonstration of Rust refusing to compile the safe version.

How Rust Prevented the Bugs

Rust prevented the bugs through a combination of ownership rules and explicit safety boundaries. Rust forbids dereferencing the raw pointer without the `unsafe` command. This forces the developer to acknowledge that they are bypassing the language's protections. Rust also did not allow me to reconstruct a vector from raw parts without `unsafe`, because doing so can easily cause double frees or aliasing violations. Underneath these restrictions are Rust's core safety principles. Ownership ensures that memory is freed exactly once. Borrowing prevents multiple mutable references or aliasing (Rust Team, 2024). Lifetimes ensure that references never outlive the data they point to. Together, these rules eliminate entire classes of bugs that C developers must guard against manually.

Comparing the Memory Models

The difference between C and Rust became clear as I worked through the project. C forces the developer to manually expand into new memory territory. This made sense in the era when every byte mattered. Rust is capable of allocating memory for the owner and keeping correctness in check automatically (The Linux Kernel Organization, 2024). It performs the same optimizations a careful C developer would, but without relying on the developer to remember every detail. C gives you total freedom, but also total responsibility. Rust gives you freedom too, but only when you explicitly ask for it and only after acknowledging the risks.

Developer Experience

Working in Rust had times where it was more verbose than C would be, but lots of times where it was less, but at the end of the day syntax is something you get used to over time. I was not planning to create a bin folder at first, but I realized I needed it to compile multiple binaries cleanly. Rust's tooling is modern and helpful, and the compiler errors are descriptive. C, on the other hand, is simple and direct, but it leaves everything up to the developer.

Reflection, Industry Context, and Future Ideas

Before this project, I had heard a lot about memory safety as an argument against C. Now that I have seen a language that can still be low level while enforcing memory safety, I understand why some people want to move away from languages without it. That said, I think I will always have a soft spot for C, and I do not think it will or should be entirely replaced by Rust. Rust adoption is increasing because it offers the same kind of optimization that low level languages like C provide, without the downsides that concern security focused developers (The Linux Foundation, 2024). At the same time, C still has advantages today. It is great for smaller projects, and many memory safety issues can be addressed with enough discipline and tooling. Rust simply makes correctness the default.

If I were interviewing for a systems job, I would summarize the project by saying that it taught me the basics of Rust programming and gave me a better grasp of how modern languages implement memory safety.

If I had more time, I would have explored tools that detect memory errors in C or expanded the project to include garbage collected languages. Memory management is a topic that interests me, and this project only deepened that interest.

References

Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language*. No Starch Press. <https://doc.rust-lang.org/stable/book/>

Loosemore, S., Stallman, R. M., McGrath, R., Oram, A., & Drepper, U. (n.d.). *The GNU C Library Reference Manual*. The GNU C Library (glibc) 2.43. <https://sourceware.org/glibc/manual/2.43/pdf/libc.pdf>

The Linux Foundation. (2024). Keynote: Linus Torvalds in conversation with Dirk Hohndel [Video]. YouTube. https://www.youtube.com/watch?v=OM_8UOPFpqE

The Kernel Development Community. (2026). *Rust — the Linux kernel documentation*. The Linux Kernel. <https://docs.kernel.org/rust/>